

Deep Learning for Audio Data

Our title

Marek Malter

*Audiokommunikation und
-Technologie*

Technische Universität Berlin
Matr.-Nr.:

Manolo Max Müller

*Audiokommunikation und
-Technologie*

Technische Universität Berlin
Matr.-Nr.:0505701

Cecylia Krauss

*Audiokommunikation und
-Technologie*

Technische Universität Berlin
Matr.-Nr.: 507099

Constantin Luca Sprenger

*Audiokommunikation und
-Technologie*

Technische Universität Berlin
Matr.-Nr.:

Abstract—This paper reports our experience with the network bending tool `nbform`. This work subscribes itself to the emerging field of network bending.

1. Introduction

Deep generative models have advanced the state of the art in audio synthesis, producing high-quality results by learning an underlying data distribution. However, fidelity to training data and so-called “learning for perfection” [BC20] have the downside that they create a certain uniformity in the generated audio. Because these models are optimized to approximate the training distribution as closely as possible, their outputs tend to be statistically homogeneous and are inherently biased towards the “average” of the data on which they were trained [MG25]. This behavior limits the model’s ability to synthesize creative, diverse, and novel audio.

In the field of creative coding and computational creativity, there is a line of research that addresses the tension between high fidelity and creative exploration. There has been a growing interest in developing methods that intentionally push generative models outside their trained distribution [CRG26]. One such approach is network bending [BLG21]. The framework is inspired by circuit bending, which is an artistic practice of repurposing electronic devices through unintended modifications of their electronic circuits, typically requiring little technical expertise[Gha04]. Network bending follows the same approach but regarding neural networks instead of electronic devices. The trained model is treated as a modifiable system, where its weights, biases, and computational graph can be altered to explore unexpected behaviors and outputs.

Network bending has been demonstrated in individual architectures, including neural vocoders based on a DDSP model [YM21] and the VAE model RAVE [MG25]. Existing implementations are typically coupled to a specific model. Applying the same bending techniques to a different architecture can be challenging, as the internal structure and naming conventions of neural audio models vary between implementations.

This work addresses this gap by proposing `nbform`, an adaptive, model-independent framework for exploratory

network bending. `nbform` is a reusable tool that allows arbitrary PyTorch-based audio models to be manipulated through a common interface. The applicability of the framework is shown in two structurally different neural audio models: RAVE [CE21], based on a variational auto-encoder (VAE) architecture, and EnCodec [Déf+22] based on an encoder–decoder architecture with quantized latent space.

1.1. Related Work

This work relies on previous research in different areas. Firstly, we will outline the practice of network bending in relation to its cultural and political implications and present some of its applications. Secondly, we will explain the technical foundations of neural audio synthesis, specifically of RAVE and EnCodec.

1.1.1. Network bending Framework. Broad et. al. [BLG21] introduced the notion of network bending as a direct manipulation of deep generative models to induce “active divergence” in the model’s output relative to the original training distribution. They propose a method where a set of transformations can be inserted as distinct layers into the computational graph during inference. Thus, there is no modification of the model’s learned weights. In contrast, Kotowski et al. [KF26] propose to intervene directly in the model’s parameter space, introducing a network bending tool that enables real-time manipulation of the weights and biases of neural audio models. Manz et al. [MG25] propose a hardware implementation of the same approach applied to RAVE, which results in a network bending instrument. McCallum and Yee-King [YM21] developed an exploratory toolkit that focuses on transformations of a DDSP-based vocoder.

From a purely machine learning perspective, all these approaches would mean creating bad results, as they actively go against the model’s intended function. However, from a creative perspective, they can be understood as methods that treat malfunction and unintended use as a creative resource. This aligns with what Kim Cascone identifies as glitch aesthetics[Cas00], where composers actively seek out those “failures” of digital technology as material for their work.

Beyond the artistic use, network bending promises political and pedagogic implications as it allows to play around with pre-trained models without significant prior knowledge or the computational costs of training them. As Kotowski et al. put it, it is a way of “exploratory intervention and repurposing” [KF26].

1.1.2. RAVE: variational Autoencoder. RAVE (Realtime Audio Variational autoEncoder)[CE21] is a deep generative model that allows fast waveform synthesis. It can generate 48kHz audio in real-time on a standard laptop CPU. Like other VAEs, it is composed of an encoder and a decoder. The encoder maps the raw audio into a lower dimensional representation, which is called the latent space. From this representation, the decoder is responsible for the reconstruction back into the audio domain. The Training of the RAVE model is split into two stages. The representation learning and the adversarial fine-tuning which aims to improve the audio quality.

1.1.3. EnCodec. EnCodec [Déf+22] is a neural audio codec for high fidelity compression of 24kHz and 48kHz stereophonic audio in real time. Its architecture consists of a convolution-based encoder and decoder, with a quantized latent space, that differs from the continuous latent space of RAVE. The encoder produces a low dimensional latent representation of the waveform audio which is then quantized using Residual Vector Quantization (RVQ). Finally, the decoder reconstructs the waveform.

1.2. Technical Design and Implementation

1.2.1. System Overview. The tool supports the transformation of pretrained neural audio models through user-defined modifications to their internal components. Its primary workflow enables users to select layers, apply parameter or structural transformations through a command-line interface, and process an input audio recording using the resulting model. Multiple transformations can be accumulated before inference, allowing users to explore combinations of modifications within and across the encoder and decoder.

1.2.2. Model and Layer Abstraction. Architecture-specific implementations of a shared `NNModel` interface handle differences between RAVE and EnCodec, allowing the transformation operators to use a common interface. These implementations provide access to encoder and decoder containers and their components, including reported layer properties such as input and output channel counts. They also specify the audio requirements of each model and provide access to supported weight and bias tensors.

The two architectures differ in how their pretrained models are loaded. For RAVE, the tool requires a training checkpoint together with its corresponding `config.bin` file. The configuration defines the model architecture, and the checkpoint supplies its learned parameters. An exported TorchScript `.ts` file alone cannot be used with the RAVE implementation for layer transformations. Consequently, the

RAVE implementation is limited to models for which these training run files are available. For EnCodec, the implementation loads the publicly available pretrained 24 kHz or 48 kHz model.

PyTorch model components are organized hierarchically: an encoder or decoder contains modules that may themselves contain further modules. The tool treats a direct child of an exposed encoder or decoder container as an intervention target. Such a layer may be an elementary operation, such as a convolution or activation, or a composite component, such as a residual block.

Each target is described by a `LayerInfo` object containing its owning model, encoder or decoder designation, position within the container, hierarchical path to the module, module class name, and reported input and output channel counts. The underlying module is retrieved through the model abstraction when needed. Provided the module hierarchy remains unchanged, the path can be used to locate the corresponding module after a model reset.

Architecture-specific inspection methods identify supported submodules and expose their weight and bias tensors. When a selected component is a composite block, its supported internal tensors are collected and transformed together; the interface does not offer separate control of each internal submodule.

1.2.3. Transformation Operators. Four operators are implemented: skipping a layer, repeating a layer, multiplying its weights and biases, and adding offsets to its weights and biases. Each operator receives a `LayerInfo` description and inserts a callable PyTorch module at the selected position in the encoder or decoder container. Structural operators change how the original component is executed; parameter operators modify its tensors before executing its original forward computation.

Let z be the input to a selected component l , and let f_l denote the function it implements. The skipping operator replaces that computation with the identity mapping:

$$\tilde{f}_l(z) = z. \quad (1)$$

The repetition operator applies the same component r times:

$$\tilde{f}_l(z) = f_l^r(z), \quad (2)$$

where r is the total number of applications of the same module and its parameters. For $r = 1$, the original computation is retained. These structural transformations require compatible dimensions: skipping passes the input directly to the following component, while repetition feeds a component’s output back into that same component. The automated analysis uses reported input and output channel counts to choose candidate components; matching channel counts alone do not establish compatibility of all tensor dimensions or internal operations.

Parameter transformations use the weight and bias tensors exposed by the model implementation. Multiplication

applies a weight scaling factor α and a bias scaling factor β independently:

$$\widetilde{W} = \alpha W, \quad \widetilde{b} = \beta b. \quad (3)$$

Addition applies independent scalar offsets γ to the weights and η to the biases:

$$\widetilde{W} = W + \gamma, \quad \widetilde{b} = b + \eta. \quad (4)$$

For a selected composite component, the operation applies to all supported weight and bias tensors exposed within it. Parameter updates take place when the transformed component is initialized. During inference, the component executes its forward computation using the modified tensors.

1.2.4. Command-Line Workflow and Transformation Composition. The command-line interface provides the primary mechanism for configuring transformations of a model. Users select a pretrained model, identify target components, and specify transformation operators and their parameters. Transformations accumulate in the working model before an input recording is processed. This enables modifications to multiple components, including combinations distributed across the encoder and decoder.

1.2.5. Audio Processing. The audio-processing mode invokes the selected model on an input recording through the shared model interface. The input consists of a waveform tensor and its sampling rate. Before inference, the waveform is converted to the sampling rate and channel configuration required by the selected model. The resulting waveform can be exported for listening and further analysis, allowing users to assess the audible effects of individual transformations and their combinations.

1.2.6. Supporting Parameter-Sweep Analysis. The parameter-sweep mode evaluates predefined transformations to identify candidate components and parameter ranges for exploration. It first computes the unmodified model’s reconstruction of a selected input recording. After each individual transformation, it compares the resulting waveform with that reconstruction. The model is restored between trials, so each comparison reflects a single intervention.

Let y_i and $\widehat{y}_i$ be sample i of the baseline reconstruction and transformed output, respectively. For N waveform samples, mean absolute error is

$$MAE(y, \widehat{y}) = \frac{1}{N} \sum_{i=1}^N |\widehat{y}_i - y_i|. \quad (5)$$

Output changes are also measured using multi-resolution short-time Fourier transform loss, which compares spectrograms computed with different analysis resolutions [YSK20]. These measures characterize waveform and spectral deviations from the baseline reconstruction. They do not establish perceptual quality or artistic usefulness; a larger deviation indicates a stronger change under the selected

metric. Likewise, an isolated-intervention ranking does not predict the effects of arbitrary transformation combinations.

The analysis returns structured results containing layer identifiers, transformation parameters, and comparison metrics, together with spectral analysis diagrams. These outputs support the inspection and communication of candidate configurations while leaving the selection and composition of transformations under user control.

2. Evaluation

The evaluation used the software and input audio listed in Table 1.

TABLE 1. SOFTWARE AND INPUT AUDIO USED FOR EVALUATION.

Python	[python 3.11.15]
PyTorch Lightning	[pytorch-lightning 1.9.0]
RAVE	[rave 2.3.1]
Encodec	[encodec 0.1.1]
Audio Input	[Sample of Slayer’s <i>Hell awaits</i>]

2.1. Conclusion and Outlook

2.1.1. Conclusion. This work presented a tool for exploring pretrained neural audio models through user-defined structural and parameter transformations. The command-line interface allows users to configure and combine transformations before processing an input recording, making model components accessible as a resource for sound manipulation. Automated parameter sweeps complement this workflow by identifying candidate layers and transformation values that can serve as starting points for further exploration.

A principal contribution is the separation of model-specific functionality from transformation logic. The `NNModel` abstraction provides a common interface for accessing model components, inspecting parameters, and performing inference. The interface can support further architectures when their structure and execution requirements can be represented by its methods. Similarly, the modular implementation of transformation operators supports the addition of further manipulations without redesigning the overall workflow.

The explored transformations produced outputs that we considered interesting for sound-design applications. This observation suggests potential for creative use, although it does not constitute a systematic assessment of perceptual quality or artistic usefulness. The current analysis quantifies differences from an unmodified reconstruction but does not establish how listeners perceive those differences.

An additional limitation concerns the granularity of intervention. Applying a transformation uniformly to the parameters exposed by an entire layer may affect several interacting features simultaneously. More selective modifications could provide finer control while preserving other aspects of the model’s behaviour. The original RAVE architecture, for example, includes distinct decoder branches for waveform

generation, a loudness envelope, and filtered noise. This provides a motivation for exploring branch-specific interventions, although it does not imply that arbitrary intermediate channels correspond to independent perceptual attributes.

2.1.2. Outlook. Future development could extend the transformation framework to finer structural levels. Potential targets include individual channels, convolutional filters and parameter subsets. Where an architecture exposes dedicated synthesis branches, these could also be manipulated separately. Such extensions would enable controlled comparisons between broad modifications and localized interventions, allowing the hypothesis that greater selectivity improves controllability to be evaluated directly.

Layer swapping represents another possible extension of the user-facing transformation workflow. Exchanging compatible components could enable exploration of how their position within a network influences the resulting audio.

The supporting analysis procedure could also be extended from predefined parameter grids to adaptive sampling. An initial set of transformation values could be sampled from a specified range and evaluated against the unmodified model reconstruction. Subsequent evaluations could concentrate on intervals exhibiting substantial changes in response or containing configurations of interest. Retaining some sampling across the full range would reduce the risk of overlooking disconnected regions with useful behaviour. The objective should be to identify informative and controllable parameter regions rather than simply maximize output deviation, since a large difference may also reflect silence, excessive amplification, or unstable behaviour.

Finally, further investigation could examine why specific transformations produce particular audible effects. Controlled parameter sweeps, inspection of intermediate activations, and measurements of properties such as loudness, spectral distribution, and temporal structure could help relate internal changes to output behaviour. Repeating these experiments across multiple recordings and checkpoints would distinguish input-dependent effects from more consistent patterns. Combined with listening evaluations, this could support more interpretable transformation controls and provide a stronger basis for guiding users toward configurations suited to their sound-design objectives.

References

- [Cas00] Kim Cascone. “The Aesthetics of Failure: “Post-Digital” Tendencies in Contemporary Computer Music”. en. In: *Computer Music Journal* 24.4 (Dec. 2000), pp. 12–18. ISSN: 0148-9267, 1531-5169. DOI: 10.1162/014892600559489. URL: <https://direct.mit.edu/comj/article/24/4/12-18/93451> (visited on 09/22/2026).
- [Gha04] Qubais Reed Ghazala. “The Folk Music of Chance Electronics: Circuit-Bending the Modern Coconut”. en. In: *Leonardo Music Journal* 14 (Dec. 2004), pp. 97–104. ISSN: 0961-1215, 1531-4812. DOI: 10.1162/0961121043067271. URL: <https://direct.mit.edu/lmj/article/63355> (visited on 09/21/2026).
- [BC20] Sebastian Berns and Simon Colton. “Bridging Generative Deep Learning and Computational Creativity”. In: *ICCC*. 2020. URL: <https://api.semanticscholar.org/CorpusID:228097446>.
- [YSK20] Ryuichi Yamamoto, Eunwoo Song, and Jae-Min Kim. “Parallel WaveGAN: A Fast Waveform Generation Model Based on Generative Adversarial Networks with Multi-Resolution Spectrogram”. In: *Proceedings of the IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. 2020. arXiv: 1910.11480. URL: <https://arxiv.org/abs/1910.11480>.
- [BLG21] Terence Broad, Frederic Fol Leymarie, and Mick Grierson. “Network Bending: Expressive Manipulation of Generative Models in Multiple Domains”. en. In: *Entropy* 24.1 (Dec. 2021), p. 28. ISSN: 1099-4300. DOI: 10.3390/e24010028. URL: <https://www.mdpi.com/1099-4300/24/1/28> (visited on 09/17/2026).
- [CE21] Antoine Caillon and Philippe Esling. *RAVE: A variational autoencoder for fast and high-quality neural audio synthesis*. en. arXiv:2111.05011 [cs]. Dec. 2021. DOI: 10.48550/arXiv.2111.05011. URL: <http://arxiv.org/abs/2111.05011> (visited on 03/17/2026).
- [YM21] Matthew Yee-King and Louis McCallum. *Studio report: sound synthesis with DDSP and network bending techniques*. 2021. URL: <https://www.tib.eu/de/suchen/id/base%3A41d00dfec7e7f7209a1ff124e7355b6ccf6c1c1>.
- [Déf+22] Alexandre Défossez et al. *High Fidelity Neural Audio Compression*. Version Number: 1. 2022. DOI: 10.48550/ARXIV.2210.13438. URL: <https://arxiv.org/abs/2210.13438> (visited on 09/21/2026).
- [MG25] Daniel Manz and Mick Grierson. “Brave: Engineering an Embedded Network-Bending Instrument, Manifesting Output Diversity in Neural Audio Systems”. eng. In: *Proceedings of the Sixteenth International Conference on Computational Creativity: ICCC 2025, Campinas, Brazil, 23-27 June*. Ed. by Hugo Gonalo Oliveira et al. Meeting Name: International Conference on Computational Creativity. Coimbra?: Association for Computational Creativity (ACC), 2025, pp. 323–328. ISBN: 978-989-54160-7-3. URL: <https://computationalcreativity.net/iccc25/papers/iccc25-manz2025brave.pdf>.
- [CRG26] Nick Collins, Vit Ruzicka, and Mick Grierson. *Remixing AIs: mind swaps, hybridity, and splicing musical models*. 2026. URL: <https://durham-repository.worktribe.com/output/1139583>.
- [KF26] Błażej Kotowski and Frederic Font. “Network Bending as Circuit-Bending Inspired Live Neural Synthesis Hacking”. In: (June 2026). ISSN:

2220-4806. DOI: 10.5281/ZENODO.20784161.
URL: <https://zenodo.org/doi/10.5281/zenodo.20784161> (visited on 07/14/2026).